SQL Injection Report

1. Lab Name

PortSwigger: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data.

2. Objective

To exploit a SQL injection vulnerability in a product category filter to retrieve hidden data from the database.

---

3. Vulnerability / Concept

SQL injection occurs when user input is directly concatenated into SQL queries without proper sanitization.

In this lab, the application uses a query like:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

The category parameter is vulnerable. By injecting the following payload, the WHERE clause is made always true and the remainder of the query is commented out:

```
' OR 1=1--
```

Key finding

The injection causes the condition to evaluate as always true, allowing hidden data to be retrieved.

---

4. Testing Approach

1. Navigated to the shopping website.
2. Clicked the “Gifts” category.
3. Observed the URL:

```
/filter?category=Gifts
```

1. Modified the URL to:

```
/filter?category=Gifts'+OR+1=1--
```

1. The resulting query became:

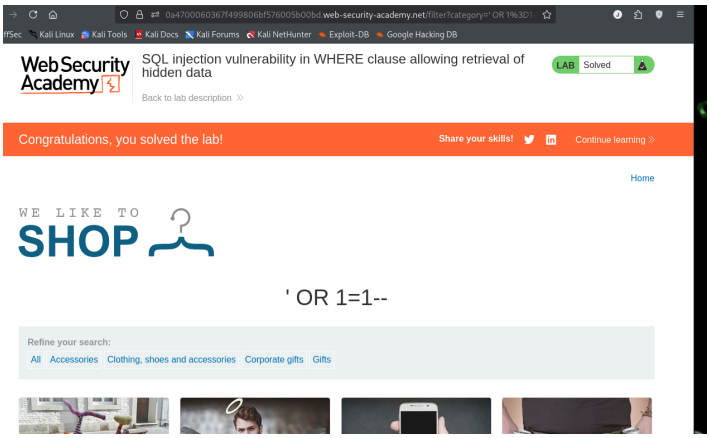
```
SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1
```

1. The OR 1=1 condition made the condition always true.
2. The -- sequence commented out the remainder of the query.
3. All products, including hidden products, appeared.
4. The lab was solved successfully.

---

5. Evidence / Screenshots

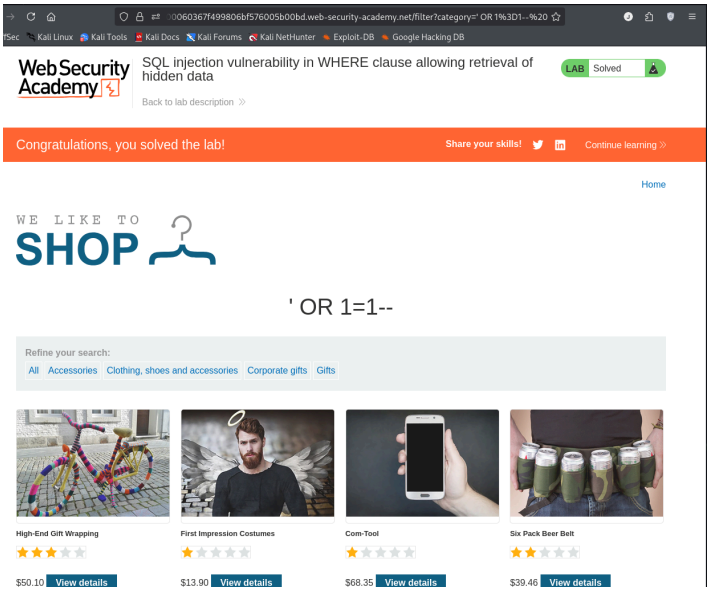
1. Lab completion showing the “Solved” banner.



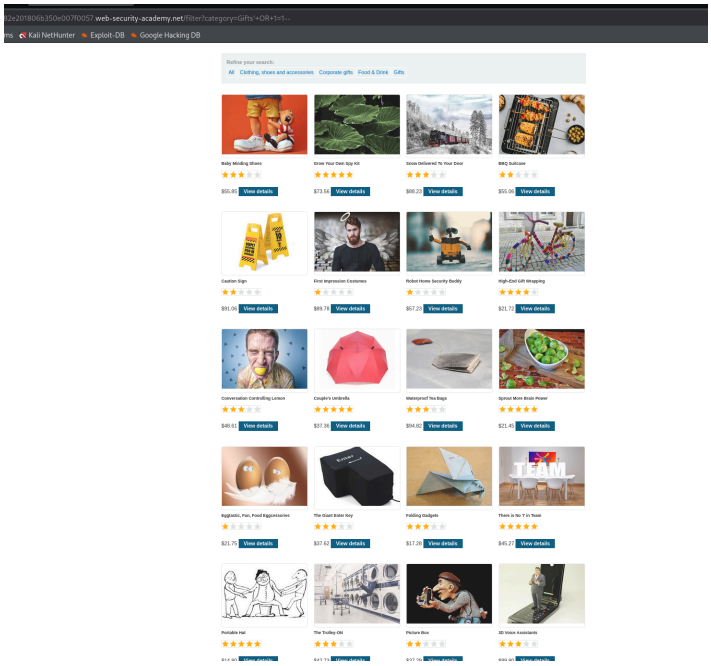
2. URL containing the SQL injection payload.



3. All products displayed, including hidden products.



4. Full-page evidence.



6. Impact

- Authentication bypass.
- Potentially unauthorized access to database contents.
- Data exfiltration.
- Data modification or deletion.
- Potential remote code execution, depending on the database configuration and application environment.

---

7. Remediation

- Use parameterized queries or prepared statements.
- Apply strict input validation and appropriate sanitization.
- Use database accounts with least privilege.
- Consider a properly configured Web Application Firewall (WAF) as a supplementary control.
- Implement proper error handling that does not expose database details.

---

8. What You Learned

- SQL injection is caused by unsanitized user input being incorporated into SQL statements.
- OR 1=1 can make a WHERE-clause condition always true.
- The -- comment sequence can bypass the remaining portion of a query.
- Parameterized queries help prevent SQL injection.
- Even a simple injection can expose hidden data.
